Arquitectura de Aplicaciones Web

Taller: Desarrollo de APIs REST con Spring Boot, Spring Data JPA y PostgreSQL

Objetivo:

Desarrollar una API REST en Spring Boot que maneje datos en PostgreSQL utilizando Spring Data JPA, implementando entidades con relaciones, consultas personalizadas y manejo de logs para depuración y monitoreo.

Duración:

3 horas

Requisitos Previos:

JDK 17 o superior instalado
PostgreSQL 14 o superior instalado y configurado
IDE como IntelliJ IDEA, Eclipse o VS Code con Extensión de Java
Postman o Swagger para probar la API
Conocimientos básicos de Java, SQL y Spring Boot

Parte 1: Configuración del Proyecto en Spring Boot con PostgreSQL

Ejercicio 1: Creación del Proyecto Spring Boot

Objetivo: Configurar un proyecto con Spring Boot, PostgreSQL y Spring Data JPA.

Paso a Paso:

1. **Ve a Spring Initializr y genera un nuevo proyecto con las siguientes opciones:**
    - **Project: Maven**
    - **Language: Java**
    - **Spring Boot Version: 3.x.x**
    - **Dependencies:**
      Spring Web
      Spring Boot DevTools
      Spring Data JPA
      PostgreSQL Driver
      Lombok (para reducir código**)**
2. **Descarga y descomprime el proyecto.**

Arquitectura de Aplicaciones Web

3. **Abre el proyecto en tu IDE (IntelliJ o Eclipse).**

Ejercicio 2: Configuración de PostgreSQL en application.properties

1. En PostgreSQL, crea una nueva base de datos llamada tienda_db.
2. En src/main/resources/application.properties, configura la conexión:

spring.datasource.url=jdbc:postgresql://localhost:5432/tienda_db

spring.datasource.username=tu_usuario

spring.datasource.password=tu_contraseña

spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

Resultado esperado: PostgreSQL configurado como base de datos de la aplicación.

Arquitectura de Aplicaciones Web

Parte 2: Creación de Entidades y Relaciones

Ejercicio 3: Creación de Entidad Producto

Objetivo: Crear una entidad Producto con atributos y anotaciones JPA.

Paso a Paso:

1. Dentro de src/main/java/com/miempresa/api, crea un paquete models.
2. Dentro de models, crea la clase Producto.java:

```
package com.miempresa.api.models;

import jakarta.persistence.*;

import lombok.*;

@Entity

@Table(name = "productos")

@Getter

@Setter

@NoArgsConstructor

@AllArgsConstructor

public class Producto {

  @Id

  @GeneratedValue(strategy = GenerationType.IDENTITY)

  private Long id;

  @Column(nullable = false)

  private String nombre;

  private double precio;

}
```

Resultado esperado: Entidad Producto mapeada a la base de datos con una tabla productos.

Arquitectura de Aplicaciones Web
Ejercicio 4: Creación de Entidad Categoría y Relación con Producto

Objetivo: Implementar una relación Uno a Muchos (1:N) entre Categoria y Producto.

Paso a Paso:

1. **Crea una nueva clase Categoria.java en el paquete models:**

```
package com.miempresa.api.models;


import jakarta.persistence.*;
import lombok.*;


import java.util.List;


@Entity
@Table(name = "categorias")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Categoria {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;


  @Column(nullable = false, unique = true)
  private String nombre;


  @OneToMany(mappedBy = "categoria", cascade = CascadeType.ALL)
```

```
  private List<Producto> productos;

}
```

2. **Modifica Producto.java para agregar la relación con Categoria:**

```
@ManyToOne

@JoinColumn(name = "categoria_id")

private Categoria categoria;
```

Resultado esperado: Relación entre Categoria y Producto establecida en la base de datos.

Parte 3: Creación de Repositorios y Consultas Personalizadas

Ejercicio 5: Creación de Repositorios

1. Dentro de src/main/java/com/miempresa/api, crea un paquete repositories.
2. Dentro de repositories, crea ProductoRepository.java:

```
package com.miempresa.api.repositories;


import com.miempresa.api.models.Producto;

import org.springframework.data.jpa.repository.JpaRepository;


import java.util.List;


public interface ProductoRepository extends JpaRepository<Producto, Long> {

  List<Producto> findByNombreContainingIgnoreCase(String nombre);

}
```

3. **Crea CategoriaRepository.java:**

```
package com.miempresa.api.repositories;


import com.miempresa.api.models.Categoria;
```

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface CategoriaRepository extends JpaRepository<Categoria, Long>
{

}
```

Resultado esperado: Repositorios creados con consultas personalizadas.

Parte 4: Creación del Controlador REST

Ejercicio 6: CRUD de Productos con Logs

1. Dentro de src/main/java/com/miempresa/api, crea un paquete controllers.
2. Dentro de controllers, crea ProductoController.java:

```
package com.miempresa.api.controllers;

import com.miempresa.api.models.Producto;
import com.miempresa.api.repositories.ProductoRepository;
import lombok.extern.slf4j.Slf4j;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/productos")
@Slf4j
public class ProductoController {
  private final ProductoRepository productoRepository;

  public ProductoController(ProductoRepository productoRepository) {
```

```
    this.productoRepository = productoRepository;
  }

  @GetMapping
  public List<Producto> listarProductos() {
    log.info("Listando todos los productos");
    return productoRepository.findAll();
  }

  @PostMapping
  public Producto crearProducto(@RequestBody Producto producto) {
    log.info("Creando producto: {}", producto.getNombre());
    return productoRepository.save(producto);
  }

  @GetMapping("/{id}")
  public Producto obtenerProducto(@PathVariable Long id) {
    log.info("Buscando producto con ID: {}", id);
    return productoRepository.findById(id).orElse(null);
  }

  @DeleteMapping("/{id}")
  public void eliminarProducto(@PathVariable Long id) {
    log.warn("Eliminando producto con ID: {}", id);
    productoRepository.deleteById(id);
  }
```

}

Resultado esperado: API con CRUD de productos y logs para depuración.

Parte 5: Pruebas y Desafíos

1. **Ejecuta la aplicación y accede a http://localhost:8080/swagger-ui.html.**
2. **Prueba los endpoints usando Swagger o Postman.**
3. **Desafío:**
    - Implementar un CRUD para Categorías.
    - Agregar métodos de búsqueda avanzada en ProductoRepository.
    - Configurar logs en archivos con logback.xml.

Resultado esperado: API REST funcional con PostgreSQL, relaciones, consultas avanzadas y logs.

Conclusión

Construimos una API REST con Spring Boot, PostgreSQL y Spring Data JPA.
Aplicamos mapeo de entidades y relaciones.
Implementamos consultas personalizadas y manejo de logs.

Siguientes pasos: Integrar Spring Security para autenticación de usuarios.